Мониторинг: от полёта вслепую к наблюдаемости

Prometheus и Kubernetes — твой штурман в полёте вслепую

Автор: Aliaksandr Tsimokhau

Курс: DevOps Pro

Маршрут

Пролог · Полёт вслепую

Признаём боль — об авариях узнаём последними

Акты 1–3 · Штурман возвращает зрение

Учимся видеть — основы, язык метрик, сбор и запросы

Акты 4–5 · Испытания прода

Проходим трудности — шум алертов и масштаб

Акт 6 → Финал · Трансформация

Закрепляем результат — VictoriaMetrics и чек-лист

Пролог

3 часа ночи. Звонит не алерт — звонит пользователь.

Как выглядит «до»

Что есть

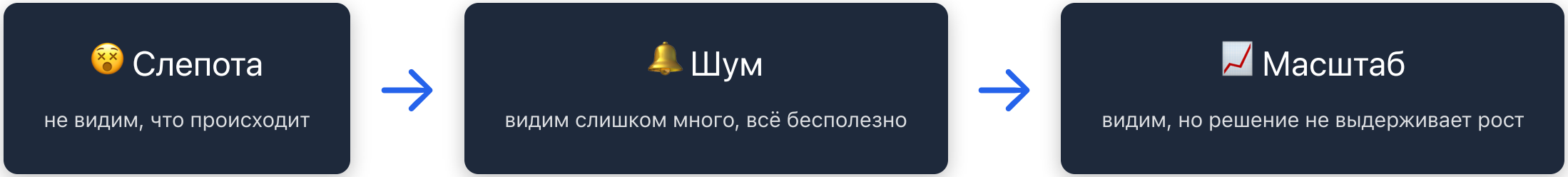
- Сервис задеплоен, healthcheck зелёный
- Логи где-то есть — если зайти на под
- «Графиков нет, но вроде работает»

Что происходит

- Латенси растёт час — никто не видит
- Память течёт — под убивает OOM ночью
- Об аварии узнаёшь из чата поддержки

> Проблема не в том, что что-то ломается. Проблема в том, что **ты узнаёшь об этом последним**.

Три боли, которые мы пройдём как путь



- Сегодня штурман проведёт тебя через все три
- Каждое испытание — отдельный акт, со своим решением
- В финале сравним «до» и «после» по-честному

Акт 1: В кабине появляется штурман

Что такое наблюдаемость и при чём здесь Prometheus

Мониторинг vs наблюдаемость

Мониторинг

Отвечает на **известные** вопросы

- «CPU выше 80%?»
- «Сервис жив?»
- Заранее заданные дашборды и алерты

Наблюдаемость

Помогает задать **новые** вопросы

- «Почему именно этот эндпоинт тормозит у 5% юзеров?»
- Исследование без передеплоя
- Метрики + логи + трейсы

> Мониторинг говорит, **что** сломалось. Наблюдаемость помогает понять, **почему**.

Три типа сигналов: метрики, логи, трейсы

Сигнал	Отвечает на вопрос	Стоимость	Где силён
Метрики	Что и насколько? (числа во времени)	Низкая	Тренды, алерты, SLO
Логи	Что именно произошло? (события)	Средняя	Детали, отладка
Трейсы	Где в цепочке вызовов?	Высокая	Латенси в микросервисах

- Начинаем с **метрик** — дешевле всего и закрывают 80% «слепоты»
- Логи и трейсы добавляем точечно, когда метрики показали *где* копать

Prometheus в одном предложении

> **Prometheus** — это база данных временных рядов, которая сама ходит за метриками к твоим сервисам, хранит их и позволяет спрашивать их языком PromQL.

- Open-source, проект CNCF (как и Kubernetes)
- Модель **pull**: Prometheus сам опрашивает цели (не они шлют ему)
- Хранит **time series** — число + метки + временная метка
- Встроенные алерты через Alertmanager

Почему команды выбирают Prometheus

Сильные стороны

- Pull-модель + service discovery — идеален для динамики k8s
- PromQL: мощные запросы по меткам
- Огромная экосистема экспортёров
- Нет внешних зависимостей: один бинарь

Честные ограничения

- Локальное хранилище = не для долгого retention «из коробки»
- Не для высокой кардинальности без оговорок
- Авторизации «из коробки» нет
- Не для логов/трейсов — только метрики

Акт 2: Учимся читать сигналы

Модель данных и типы метрик

Модель данных: всё держится на метках

```
http_requests_total{method="POST", handler="/login", status="500"}  →  42
└──── имя метрики ────┘ └──────────── метки (labels) ─────────────┘    значение
```

- Каждая уникальная комбинация **имя + метки** = отдельный time series
- Метки позволяют резать метрику: по эндпоинту, статусу, инстансу
- Та же модель — фундамент и для запросов, и для алертов

> Метки — это сила Prometheus. И, как увидим в Акте 5, его главная ловушка.

Четыре типа метрик

Тип	Поведение	Классический пример
Counter	только растёт (сбрасывается при рестарте)	`http_requests_total`
Gauge	растёт и падает	`memory_usage_bytes`
Histogram	распределение по корзинам	`request_duration_seconds`
Summary	квантили, считает клиент	`rpc_duration_seconds`

- Выбор типа — это выбор вопроса, на который метрика отвечает
- Разберём каждый на следующих слайдах с правильным запросом

Counter — «сколько всего произошло»

```
# Не смотри на сырой counter — смотри на скорость
rate(http_requests_total[5m])

# RPS по статусам — видно всплеск ошибок
sum by (status) (rate(http_requests_total[5m]))
```

- Только растёт; интересна **скорость**, а не абсолютное значение
- `rate()` корректно обрабатывает рестарты (сброс counter)
- Вопрос героя: «как часто это происходит?»

Gauge — «сколько прямо сейчас»

```
# Текущее использование памяти
container_memory_working_set_bytes{pod=~"checkout-.*"}

# Тренд: растёт ли утечка
deriv(container_memory_working_set_bytes{pod=~"checkout-.*"}[30m]) > 0
```

- Мгновенное значение: память, очередь, число соединений
- К gauge применяют `avg/max/min`, а не `rate`
- Вопрос героя: «утечка памяти из пролога — её видно?» — да, вот она

Histogram — «как распределяется латенси»

```
# 95-й перцентиль латенси за 5 минут
histogram_quantile(
  0.95,
  sum by (le) (rate(http_request_duration_seconds_bucket[5m]))
)
```

- Раскладывает наблюдения по корзинам (`_bucket` , `_sum` , `_count`)
- Перцентили считаются **на сервере** — гибко, можно агрегировать
- Основа для **SLO**: «95% запросов быстрее 300мс»

Summary — квантили на стороне клиента

Когда подходит

- Нужны точные квантили без настройки корзин
- Один инстанс, без агрегации между подами

Почему чаще берут Histogram

- Summary **нельзя агрегировать** между инстансами
- Квантиль фиксируется при инструментировании
- В k8s с N подами это критично

> Практика: в Kubernetes по умолчанию выбирай **Histogram**, Summary — для редких точечных случаев.

Практическое правило для меток

✅ Хорошие метки

- Низкая кардинальность: `method`, `status`, `region`
- Конечный, предсказуемый набор значений

❌ Опасные метки

- `user_id`, `email`, `request_id`
- `url` с параметрами, timestamp
- Всё «бесконечное»

> Каждое уникальное значение метки = новый time series. Метка с `user_id` = миллионы рядов.

Пример: инструментируем сервис

```python
from prometheus_client import Counter, Histogram, start_http_server

REQS = Counter("http_requests_total", "Всего запросов", ["method", "status"])
LATENCY = Histogram("http_request_duration_seconds", "Латенси", ["handler"])

@LATENCY.labels(handler="/checkout").time()
def handle_checkout(req):
    REQS.labels(method=req.method, status="200").inc()
    ...

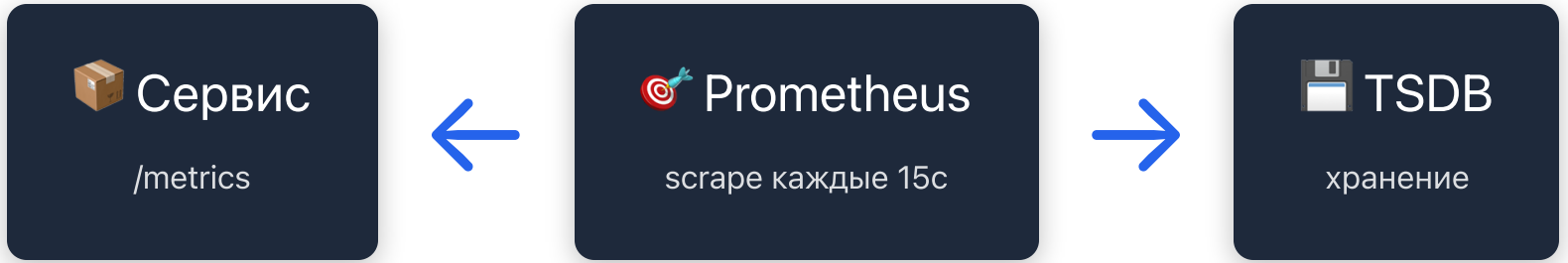
start_http_server(8000)   # метрики на :8000/metrics
```

- Приложение **отдаёт** `/metrics`, Prometheus **забирает** их сам
- Метки заданы осознанно — никакой бесконечной кардинальности

Акт 3: Штурман сам снимает показания

Сбор (scraping), service discovery и PromQL

Pull-модель: кто кого опрашивает



Плюсы

- Prometheus знает, что цель «жива» (up=0 — уже алерт)
- Легко в динамике k8s: появился под — нашли и опросили
- Цель не обязана знать про Prometheus

Компромиссы

- Краткоживущие задачи (cron) — через Pushgateway
- Цель должна быть сетево достижима
- Интервал scrape = гранулярность данных

Экспортёры и service discovery

Экспортёры

Переводят «чужие» метрики в формат Prometheus

- `node_exporter` — хост (CPU, диск, сеть)
- `kube-state-metrics` — состояние объектов k8s
- `postgres_exporter`, `redis_exporter`…

Service discovery

Prometheus находит цели автоматически

- Kubernetes SD: поды, сервисы, эндпоинты
- Появился под — он в мониторинге без правок конфига
- Метки из k8s → метки метрик

PromQL: алфавит запроса

```
# имя метрики — все её ряды
http_requests_total

# фильтр по меткам: = равно, != не равно
http_requests_total{app="checkout", status="500"}

# =~ — совпадение по шаблону (regex), !~ — не совпадает
http_requests_total{status=~"5.."}
```

- Базовый запрос = **имя метрики** + **фильтр по меткам** в `{ }`
- Четыре оператора отбора: `=` `!=` `=~` `!~`
- Результат — *мгновенный вектор*: одно значение на каждый ряд

Мгновенный вектор vs диапазон

```
# мгновенный вектор: значение «прямо сейчас»
container_memory_working_set_bytes{pod=~"checkout-.*"}

# диапазонный вектор: все точки за 5 минут (нужен для rate)
http_requests_total[5m]
```

- **Мгновенный** (instant) — одно значение на ряд; для gauge и порогов
- **Диапазонный** (range) `[5m]` — окно точек; нужен функциям вроде `rate()`
- Counter без диапазона бесполезен — скорость считается по окну

Три кирпича запроса

Кирпич	Что делает	Пример
`rate(...[5m])`	скорость counter в секунду	`rate(http_requests_total[5m])`
`sum by (label)`	агрегирует ряды по метке	`sum by (status) (rate(http_requests_total[5m]))`
арифметика	доли и пороги	`errors / total > 0.05`

- Почти любой полезный запрос — комбинация этих трёх
- Из них собираются и дашборды, и алерты, и SLO

PromQL начинается с вопроса прода

Вопрос инженера	PromQL
Сколько ошибок в секунду?	`sum(rate(http_requests_total{status=~"5.."}[5m]))`
Доля ошибок?	`... / sum(rate(http_requests_total[5m]))`
Какие поды близки к лимиту памяти?	`container_memory_working_set_bytes / container_spec_memory_limit_bytes > 0.9`
p95 латенси по сервису?	`histogram_quantile(0.95, sum by (le, app) (rate(...bucket[5m])))`

- Сначала формулируй вопрос словами → потом переводи в PromQL
- Думай в терминах **скоростей** и **долей**, а не сырых чисел

Grafana: вопросы прода на одном экране

Как устроено

- Prometheus подключён как **datasource**
- Каждая панель = один PromQL-запрос
- Переменные `$namespace`, `$pod` вместо копий дашборда
- **Explore** для запросов, под которые панели ещё нет

Дашборд checkout (RED + память)

- **Rate**: запросы в секунду
- **Errors**: доля 5xx
- **Duration**: p95 латенси
- **Saturation**: `working_set / limit` памяти

> Дашборд хранит вопросы, заданные заранее. Explore нужен для новых вопросов во время инцидента.

Recording rules vs alerting rules

Recording rules

- Заранее считают тяжёлый запрос
- Сохраняют как новую метрику
- Дашборды грузятся быстрее
- `job:http_errors:rate5m`

Alerting rules

- Условие → если истинно N минут → алерт
- Уходит в Alertmanager
- `for: 5m` гасит мгновенные всплески

> Recording rule — «посчитай заранее». Alerting rule — «разбуди меня, если…».

Акт 4: Первое испытание — шум

Видеть всё — это тоже слепота

Новая боль: алерт-усталость

«До» этого акта

- 200 алертов в день
- Половина — «CPU 81% на 10 секунд»
- Команда мьютит канал
- Реальную аварию пропускают в шуме

Чего хотим

- Алерт = «надо действовать сейчас»
- Меньше алертов, но каждый важен
- Доверие к каналу восстановлено

> Сто алертов, которые игнорируют, хуже, чем три, на которые реагируют.

Хороший алерт — это призыв к действию

- **Симптом, а не причина:** алерти на «пользователю плохо», не на «CPU высокий»
- **Actionable:** получатель знает, что делать прямо сейчас
- **С контекстом:** ссылка на дашборд и runbook в самом алерте
- **С порогом терпения:** `for: 5m` отсекает мгновенные всплески

```
- alert: HighErrorRate
  expr: job:http_errors:ratio5m > 0.05
  for: 10m
  annotations:
    summary: "5xx > 5% на {{ $labels.app }} 10 минут"
    runbook: "https://wiki/runbooks/high-error-rate"
```

Alertmanager: из сорока сообщений одно

Группировка

- `group_by` склеивает похожие алерты
- Упал узел: одно сообщение вместо сорока

Подавление

- `inhibit_rules`: важный алерт глушит следствия
- `NodeDown` скрывает алерты подов на этом узле

Тишина

- Silence на время работ
- Задаётся заранее, с автором и сроком

Маршрутизация: `critical` будит дежурного, `warning` уходит в канал команды.

> Prometheus решает, *что* случилось. Alertmanager решает, *кому*, *когда* и *сколько раз* об этом сказать.

Пример: маршрутизация алертов

```yaml
route:
  receiver: team-slack            # по умолчанию: канал команды
  group_by: [alertname, namespace]
  group_wait: 30s                 # ждём похожие алерты перед первым сообщением
  group_interval: 5m
  repeat_interval: 4h             # не повторяем чаще раза в 4 часа
  routes:
    - matchers:
        - severity="critical"
      receiver: oncall-pager      # будит дежурного

inhibit_rules:
  - source_matchers:
      - alertname="NodeDown"
    target_matchers:
      - severity="warning"
    equal: [node]                 # глушим только на том же узле
```

- Конфиг лежит в git и проходит ревью, как алерты
- В Kubernetes его можно описать через CRD `AlertmanagerConfig`

SLO превращают метрики в обещания

Идея

- **SLI** — что меряем (доля успешных запросов)
- **SLO** — цель (99.9% за 30 дней)
- **Error budget** — сколько можно «сломать»

Зачем герою

- Алертим на **сжигание бюджета**, не на каждый чих
- Прекращаем спорить «это инцидент или нет»
- Histogram из Акта 2 даёт SLI по латенси

> Алерт «сжигаем error budget слишком быстро» в разы тише и полезнее, чем «CPU 80%».

Акт 5: Масштаб ломает наивный сетап

Когда один Prometheus.yaml уже не спасает

Зачем нужен Prometheus Operator

Боль без оператора

- Конфиг scrape правится руками
- Новый сервис → правка `prometheus.yaml` → reload
- Десятки команд дерутся за один файл

С оператором

- Мониторинг описывается через **CRD**
- Команда сама кладёт `ServiceMonitor` рядом с сервисом
- Оператор собирает конфиг и перезагружает Prometheus

> Оператор — это паттерн «оператора-человека», зашитый в контроллер: он знает, *как* эксплуатировать Prometheus в Kubernetes.

Ключевые CRD стека

CRD	Зачем
`Prometheus`	сам инстанс Prometheus (реплики, retention, ресурсы)
`ServiceMonitor`	как скрейпить набор сервисов
`PodMonitor`	как скрейпить поды напрямую
`PrometheusRule`	recording- и alerting-правила
`Alertmanager`	инстанс и конфиг алертинга

- Мониторинг становится **частью кода** сервиса, а не отдельной епархией

Пример: ServiceMonitor

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: checkout
  labels: { release: kube-prometheus-stack }
spec:
  selector:
    matchLabels: { app: checkout }
  endpoints:
    - port: metrics
      interval: 15s
      path: /metrics
```

- Кладётся рядом с сервисом, в его же репозиторий
- Оператор увидит лейбл и добавит цель в scrape автоматически

Рекомендуемая форма стека в Kubernetes

Платформенный слой

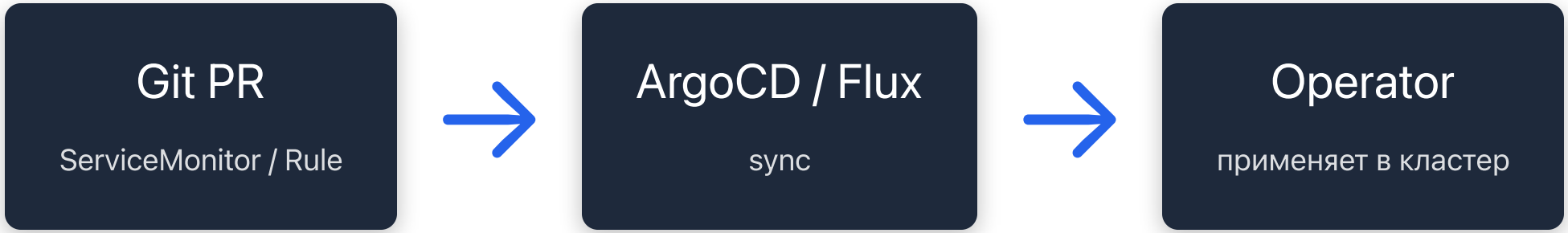
- `kube-prometheus-stack` (Helm)
- Prometheus + Alertmanager + Grafana
- node-exporter, kube-state-metrics
- Готовые дашборды и правила

Прикладной слой

- Сервис отдаёт `/metrics`
- Свой `ServiceMonitor`
- Свои `PrometheusRule`
- Дашборд как код

> Платформа даёт фундамент, команды владеют своими сигналами. Это масштабируется организационно.

GitOps: конфигурация мониторинга как код



- Любое изменение мониторинга — через review и историю в git
- Откат алерта = `git revert`, а не «кто-то поправил в UI ночью»
- Тот же процесс, что и для приложений

Подводные камни прода (1/2)

💥 Взрыв кардинальности

- Метка с `user_id` / `request_id` → миллионы рядов
- Память Prometheus улетает, всё тормозит
- **Лечение:** правило меток из Акта 2; следи за `prometheus_tsdb_head_series`

🗄️ Retention и долгое хранение

- Локальный диск = дни/недели, не годы
- Нужны квартальные тренды и аудит
- **Лечение:** Thanos / VictoriaMetrics (Акт 6)

Подводные камни прода (2/2)

🏷️ Конфликт меток (label clash)

- Метка из релейбла затирает метку приложения
- Данные «склеиваются» или теряются
- **Лечение:** соглашения об именах, аккуратный relabel

🔓 Открытый доступ

- UI/API Prometheus без авторизации = утечка топологии
- **Лечение:** ingress + auth, сетевые политики, не светить наружу

> Большинство аварий мониторинга — не баги Prometheus, а отсутствие дисциплины меток, retention и доступа.

Акт 6: Штурман получает новые приборы

VictoriaMetrics — когда Prometheus упёрся в масштаб

VictoriaMetrics: тот же мир, меньше боли

Что решает

- В разы меньше памяти на тех же данных
- Долгое хранение «из коробки»
- Лучше держит высокую кардинальность
- Совместим с PromQL (MetricsQL ⊃ PromQL)

Почему переход мягкий

- Тот же протокол scrape и форматы
- Дашборды Grafana работают как есть
- Свой оператор с аналогами CRD

> Герой не учится заново — у штурмана просто приборы точнее там, где было больно.

Две формы поставки

Single (одна нода)

- Один бинарь: приём, хранение, запросы
- Покрывает большинство команд
- Проще эксплуатировать
- Вертикальное масштабирование

Cluster

- `vmstorage` / `vminsert` / `vmselect`
- Горизонтальное масштабирование
- Мультитенантность
- Для высокой нагрузки и многих команд

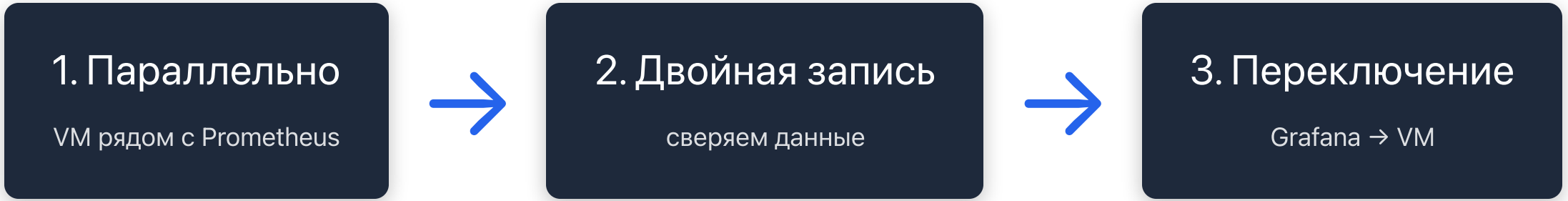
> Начинай с Single. Переходи на Cluster, только когда упрёшься в одну ноду.

Сопоставление CRD: знакомое под новым именем

Prometheus Operator	VictoriaMetrics Operator
`Prometheus`	`VMSingle` / `VMCluster`
`ServiceMonitor`	`VMServiceScrape`
`PodMonitor`	`VMPodScrape`
`PrometheusRule`	`VMRule`
`Alertmanager`	`VMAlertmanager`

- Миграция = в основном переименование ресурсов
- Есть авто-конвертация существующих `ServiceMonitor` → `VMServiceScrape`

Миграция без большого взрыва



- Включаем авто-конвертацию CRD — старые `ServiceMonitor` продолжают работать
- Сверяем графики на двух источниках, прежде чем убрать Prometheus
- Откат всегда возможен, пока оба живы

Финал: каким стал герой

От полёта вслепую — к осознанной наблюдаемости

До → После

Боль	Было (до)	Стало (после)
😵 Слепота	об авариях узнаём от юзеров	метрики + алерты видят раньше нас
🔔 Шум	200 алертов, канал замьючен	actionable-алерты по SLO/симптомам
📈 Масштаб	один `prometheus.yaml`, OOM	Operator + GitOps + VictoriaMetrics
🔒 Доступ	открытый UI	auth, сетевые политики
🗄️ История	дни на локальном диске	долгое хранение и тренды

Что это даёт в цифрах

↓ MTTR

находим причину минутами, а не часами

↓ шум

меньше алертов — выше доверие

↑ сон

3 ночи будит только важное

Чек-лист best practice

- ✅ Алерти на **симптомы** (страдает пользователь), не на причины
- ✅ Держи метки под контролем — никакого `user_id` в labels
- ✅ Histogram по умолчанию; смотри `rate()`, а не сырые counters
- ✅ Мониторинг как код: CRD + GitOps, ревью и откаты
- ✅ Спланируй retention и доступ **до** инцидента, не после
- ✅ Перерос Prometheus — апгрейд на VictoriaMetrics, а не костыли

Приложение: полная картина

Логи (EFK) и трейсы (Jaeger) — когда метрик мало

EFK: централизованные логи

Компоненты

- **Elasticsearch** — хранение и поиск
- **Fluentd** — сбор и доставка логов
- **Kibana** — поиск и визуализация

Зачем

- Метрика показала всплеск 5xx → идём в логи за стектрейсом
- Один поиск по всем подам вместо `kubectl logs` на каждом

> Метрики говорят «где горит». Логи показывают «что именно сгорело».

Loki и Alloy: логи рядом с метриками

Компоненты

- **Loki** — хранит логи и индексирует только метки
- **Alloy** — собирает логи с подов и отправляет в Loki (замена Promtail)
- **Grafana** — один Explore для метрик и логов

Запрос (LogQL)

```
sum(rate({namespace="demo-logs"} | json | level="error" [1m]))
```

Ошибки в секунду, посчитанные из логов

> Метрика показывает *где* растёт ошибка, LogQL показывает *что* именно писало приложение.

Jaeger: распределённый трейсинг

Идея

- **Trace** — путь одного запроса через сервисы
- **Span** — один шаг (вызов БД, RPC)
- Видно, *где* в цепочке потерялись миллисекунды

Когда незаменим

- Микросервисы: «медленно», но непонятно где
- Метрики дали симптом, трейс даёт точное место

> Трейсинг отвечает на вопрос, который метрики задать не могут: **в каком сервисе** именно тормоз.

LAB 9

Monitoring and Observability

с Prometheus и Grafana

Твоя очередь стать героем: разверни стек и поймай первую аварию до пользователей.